23CSE455 Design Patterns
Case Study Report

Design Pattern Identification and Evolution in a React and Node.js-Based Restaurant Management Application

Team: B1 **Date:** 11 October 2026

Application: Restaurant POS System **Team repository:** DP_Project

Original repository: restaurant-pos-system

Team members

Roll number	Student name	Batch	Contribution area
AM.SC.U4AIE23032	Surya Dingari	AIE-A	Facade; Chain of Responsibility
AM.SC.U4AIE23110	Vijay Reddy	AIE-B	Strategy; Observer
AM.SC.U4AIE23057	Srivarma	AIE-A	Adapter; State
AM.SC.U4AIE23039	Bruhadesh	AIE-A	Factory Method; Decorator
AM.SC.U4AIE23053	Teja	AIE-A	Command; Template Method

Members should confirm that the contribution areas reflect their completed work.

1 Application and architecture

The selected application is a restaurant management and point-of-sale system. Its documented technology stack includes React and TypeScript on the frontend, Node.js and Express on the backend, Prisma for database access, and Socket.IO for real-time communication. The application supports restaurant workflows such as menu access, order creation, payment handling and kitchen updates.

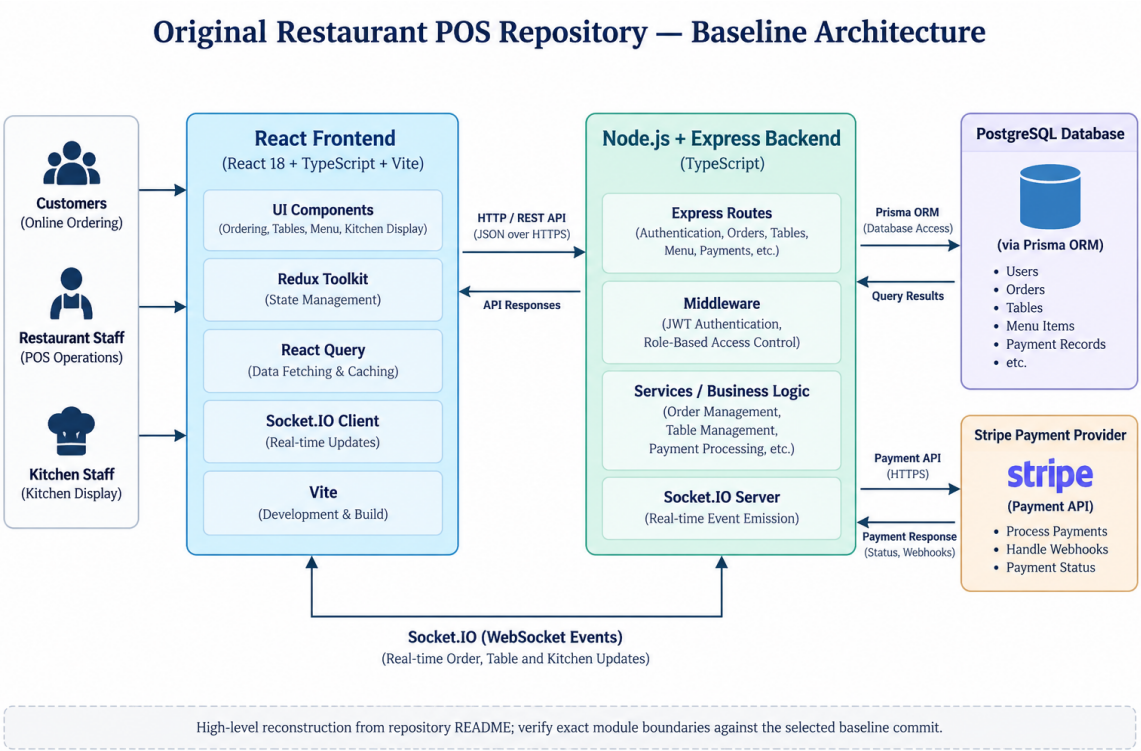


Figure 1: High-level application architecture reconstructed from repository documentation. This is a conceptual representation, not a commit-verified source diagram.

Requests travel from the React interface to Express routes, through middleware and application services. Services use Prisma for persistence and may publish real-time updates through Socket.IO.

Baseline details: The inspected local Git baseline was commit accessed on 11 October 2026. The original repository is available at https://github.com/iono-such-things/restaurant-pos-system. No root-level license file was found during inspection; therefore, the repository's license status remains unverified. The documented technology stack includes React, Express, Prisma, Socket.IO and TypeScript.

Verification note: The audit recorded 28/28 local assertions passing. These assertions do not establish full-stack HTTP, database or live Socket.IO integration.

2 Part 1: Pattern discovery

Pattern candidates were identified by inspecting source responsibilities, dependencies, method calls and collaborations. Framework usage alone was not treated as proof of a GoF pattern.

Pattern	Candidate instance	Count	Status
Facade	Authentication service	1	Confirmed
Facade	Order creation service	1	Confirmed
Observer-like	Restaurant/kitchen event collaboration	1	Confirmed as event-based collaboration
Chain of Responsibility	Express authentication middleware	1	Uncertain
Active Record	Prisma models	1	Rejected

Authentication Facade: `AuthService.login` in `apps/backend/src/services/auth.service.ts` coordinates authentication operations involving Prisma, bcrypt and JWT. It provides callers with one service entry point for a multi-step workflow.

Order creation Facade: `OrderService.createOrder` in `apps/backend/src/services/order.service.ts` coordinates order logic, persistence and event publication.

Observer-like collaboration: `apps/backend/src/websocket.ts` and the frontend `KitchenDisplay.tsx` participate in real-time order and kitchen updates. This is described as Observer-like publish/subscribe behavior; the evidence does not establish a separate canonical GoF Subject/Observer class hierarchy.

Uncertain/rejected candidates: Express middleware in `apps/backend/src/middleware/auth.ts` uses `next()`, but a custom handler hierarchy was not established. Therefore, Chain of Responsibility remains uncertain. Active Record is rejected because persistence is performed through Prisma Client rather than model-owned persistence methods.

The inspected baseline therefore supports two Facade instances and one Observer-like event collaboration. Source line ranges should be recorded for each classification, and the same classifications should be used consistently in the evidence files and final report.

3 Part 2: Evolution and refactoring

CH01 – Online ordering. Public menu and online-order routes allow customers to submit orders. The service validates input, calculates prices and publishes order events. The change extends the order-service workflow, but HTTP execution and database-backed order creation have not been verified end to end. A frontend preview fallback is not proof of a persisted order.

CH02 – Kitchen workflow. Orders are mapped to stations such as cold, hot and bar, with station-specific events published to connected clients. Nine local mapping assertions passed. Live Socket.IO delivery and station-room behavior remain unverified.

CH03 – Payment strategies. `IPaymentStrategy`, concrete strategies and `PaymentContext` sepa-

rate payment selection from the caller. Local assertions cover strategy selection, but card and mobile implementations simulate success; real payment-provider integration is not established.

Change	Design effect	Remaining verification
CH01	Extends order service and event flow	HTTP and database integration
CH02	Adds station mapping and event routing	Live Socket.IO delivery
CH03	Introduces Strategy-based payment selection	Persistence, failure handling and provider integration

The 28/28 local assertions are not full-stack integration tests. The evolved working tree was uncommitted, and complete behavior-preservation tests and before/after LOC or complexity measurements were not established. Record an immutable evolved commit and report only measured metrics. A refactoring should be described as behavior-preserving only when relevant tests support that claim.

4 Part 3: Cross-language comparison

Ten patterns were implemented as standalone demonstrations in Java, Python, JavaScript and C++: Adapter, Command, Chain of Responsibility, Decorator, Facade, Factory Method, Observer, State, Strategy and Template Method.

Language	Programs	Passed/verified	Result
Java	10	10	Executed successfully
Python	10	10	Executed successfully
JavaScript	10	10	Executed successfully
C++	10	10	Compiled and executed successfully

All 40 standalone programs were recorded as successfully executed: 10 programs in each of the four languages. The recorded environments included Node.js 22.21.1, CPython 3.10.11 and OpenJDK/javac 11.0.30.

Source LOC and timing evidence are in `data/cross_language.csv` and `Part3/evidence/reproduced_2026-10-1`
The timing procedure used a warm-up and five process-time measurements per runnable example. These results characterize the supplied demonstrations and environment, not the languages in general. Maximum function cyclomatic complexity remains unavailable and should be measured with a consistent tool and settings before numerical comparisons are reported.

The examples compare interfaces, inheritance, composition and object creation. They are separate from the patterns confirmed in the restaurant application.

5 Findings, limitations and references

The baseline supports two Facade instances and one Observer-like event collaboration. The evolution study introduces online ordering, kitchen station events and a Strategy-based payment structure.

Remaining limitations include incomplete full-stack integration testing, simulated payment success, missing cyclomatic-complexity measurements, incomplete behavior-preservation evidence and incomplete LLM experiment records.

Any LLM-assisted experiment should include the actual model and version, access tier, date, exact prompt, relevant output, source commit and human verification. Missing details must not be invented.

Before submission: verify the baseline commit and license; record source line ranges for the pattern classifications; reconcile the uncertain Chain of Responsibility candidate across all records; commit the evolved application; retain accurate test and complexity limitations; and verify team contributions.

1. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

2. Original application repository: https://github.com/iono-such-things/restaurant-pos-system.
3. Team repository: https://github.com/SrivarmaBattini/DP_Project.
4. React documentation: https://react.dev/.
5. Express documentation: https://expressjs.com/.
6. Prisma documentation: https://www.prisma.io/docs.
7. Socket.IO documentation: https://socket.io/docs/v4/.